

Simple Datatypes - Answers

Note: Code solutions are provided in the [files/svar/](#) directory. This file contains explanations, PBL answers, and references to the solution files.

Answer: Binary Numbers

Solution: Mathematical calculation (no code file needed)

Answers:

- Number of unique numbers with 3-bit: **8**
- Highest number (base-10) with 3-bit: **7**

Calculation:

- Each bit can be 0 or 1 → 2 possibilities per bit
- 3 bits: $2 \times 2 \times 2 = 2^3 = 8$ unique combinations
- Binary combinations: 000, 001, 010, 011, 100, 101, 110, 111
- 111 binary = $1 \times 4 + 1 \times 2 + 1 \times 1 = 7$ decimal

PBL Answer: For a 4-bit binary number:

- Number of unique numbers: **16** ($2^4 = 16$)
 - Highest number: **15** ($2^4 - 1 = 15$)
 - Binary: 1111 = 15 decimal
-

Answer: ASCII Characters

Solution file: [files/svar/ascii_svar.c](#)

Answers:

- 0x55: **U** (ASCII 85)
- 0x21: **!** (ASCII 33)
- 0x33: **3** (ASCII 51)

Expected Output:

```
0x55: U
0x21: !
0x33: 3
```

PBL Answer: Printing with **%d** instead of **%c**:

- 0x55: 85 (prints the decimal value)

- 0x21: 33 (prints the decimal value)
- 0x33: 51 (prints the decimal value)

Explanation:

- `%c` interprets the value as an ASCII character code
- `%d` interprets the value as a decimal integer
- Both print the same underlying value, but in different representations

Answer: ESP32 Unicode

Solution file: [files/esp32_unicode/esp32_unicode.ino](#)

Answers:

String	Output	Language	Danish Translation	Size (bytes)
string1	Hello	English	Hej	6
string2	你好	Chinese	Hej	7
string3	こんにちは	Japanese	Hej	16
string4	Здравствуйте	Russian	Hej	25
string5	💩	Emoji	Heh	5

PBL Answer: Characters take different numbers of bytes because:

1. **ASCII** uses 1 byte per character (7 bits for the character + 1 bit for parity/checking)
2. **Unicode** (UTF-8 encoding) uses variable-length encoding:
 - 1 byte: Basic Latin characters (same as ASCII)
 - 2-3 bytes: Chinese, Japanese, Cyrillic characters
 - 4 bytes: Emojis and other special symbols
3. **Difference:**
 - ASCII: Fixed 1 byte, limited to 128 characters
 - Unicode: Variable length (1-4 bytes), supports all characters worldwide
 - UTF-8 (the encoding used here) is backward-compatible with ASCII

Answer: Patient Data Types

Solution file: [files/svar/patient1_svar.c](#)

Expected Output:

```
id: 311
Marital Status: Married
Gender: Male
```

Age: 37
Height: 1.82M, Weight: 82.0Kg, bmi: 24.8

PBL Answer: Extended program with BMI classification:

```
#include <stdio.h>

int main() {
    int id = 311;
    int age = 37;
    char gender = 'M';
    char marital_status = 'G';
    float weight = 82.0;
    float height = 1.82;
    float bmi = weight / (height * height);

    printf("id: %d\n", id);

    if (marital_status == 'G') {
        printf("Marital Status: Gift\n");
    } else if (marital_status == 'S') {
        printf("Marital Status: Single\n");
    }

    if (gender == 'M') {
        printf("Gender: Mand\n");
    } else if (gender == 'F') {
        printf("Gender: Kvinde\n");
    }

    printf("Age: %d\n", age);
    printf("Height: %.2fM, Weight: %.1fKg, bmi: %.1f\n", height, weight,
bmi);

    // BMI classification
    if (bmi < 18.0) {
        printf("BMI Category: Underweight\n");
    } else if (bmi >= 18.0 && bmi < 25.0) {
        printf("BMI Category: Normal weight\n");
    } else {
        printf("BMI Category: Overweight\n");
    }

    return 0;
}
```

Note: The original answer file uses `int` for `marital_status` and switch statements. The above is a cleaner version using `char` as specified in the question.


```
int INT_SZ_BITS = sizeof(int) * 8;
unsigned int mask = 1 << (INT_SZ_BITS - 1);

for (int tal = -15; tal < 15; tal++) {
    printf("Tal: [%i]_10, [", tal);
    for (int i = 0; i < INT_SZ_BITS; i++) {
        (tal & (mask >> i)) ? printf("1") : printf("0");
    }
    printf("]_2, [0x%x]_16\n", tal);
}
return 0;
}
```

Note: The original answer file had a bug: `(tal & mask >> i)` should be `(tal & (mask >> i))` to correctly shift the mask before the AND operation.

File References

All answer files are located in:

- Binary Numbers: Mathematical calculation (no code file)
- ASCII Characters: [files/svar/ascii_svar.c](#)
- ESP32 Unicode: [files/esp32_unicode/esp32_unicode.ino](#)
- Patient Data Types: [files/svar/patient1_svar.c](#)
- Binary to Decimal: [files/svar/bin2tal_svar.c](#)
- Decimal to Binary: [files/svar/tal2bin_svar.c](#)